

Documentation du Projet CycleGAN

Exauce Kenny Tshibuabua

January 13, 2025

Introduction

Dans ce projet, nous avons exploré l'utilisation d'un modèle d'apprentissage profond appelé **CycleGAN** pour transformer des images entre deux domaines visuels non alignés. Par exemple, nous avons transformé des photos réelles en peintures de Monet.

L'objectif principal de CycleGAN est d'apprendre deux transformations :

- De **domaine A** (par exemple, des photos) à **domaine B** (par exemple, des peintures).
- De **domaine B** à **domaine A**, tout en maintenant la cohérence cyclique.

Objectif

Le but du projet est de créer un pipeline capable de :

1. Charger et prétraiter des données d'images de différents styles (photos et peintures).
2. Construire un modèle **CycleGAN** pour effectuer une transformation de style.
3. Générer des transformations fluides entre ces styles, visualisables sous forme de GIF ou vidéo.

Comment fonctionne un CycleGAN ?

Principe de base des GANs

Un **GAN** (Generative Adversarial Network) est composé de deux réseaux neuronaux :

- **Générateur (G)** : Il génère des données qui imitent le domaine cible.
- **Discriminateur (D)** : Il distingue entre les données réelles et celles générées.

Extension à CycleGAN

CycleGAN ajoute une contrainte de *cohérence cyclique*. Cela signifie que :

- Lorsqu'une image passe de $A \rightarrow B$ et revient $B \rightarrow A$, elle doit rester similaire à l'originale.
- Cette contrainte est appliquée via une **perte cyclique**.

Fonction de perte de CycleGAN

Le modèle utilise trois fonctions de perte :

- **Perte GAN** : Encourager G à produire des données réalistes.
- **Perte cyclique** : Garantir que $A \rightarrow B \rightarrow A$ ou $B \rightarrow A \rightarrow B$ conserve l'identité.
- **Perte d'identité** : Maintenir la couleur et les textures similaires.

$$\mathcal{L}_{\text{CycleGAN}} = \mathcal{L}_{\text{GAN}} + \lambda_{\text{cycle}} \mathcal{L}_{\text{cycle}} + \lambda_{\text{identity}} \mathcal{L}_{\text{identity}}$$

Étapes du projet

1. Préparation des données

Nous avons utilisé deux ensembles d'images :

- **monet_jpg** : Contient 300 peintures de Monet.
- **photo_jpg** : Contient 1000 photos réelles.

Les étapes incluent :

- Chargement des données depuis Google Drive.
- Redimensionnement des images à 256×256 .
- Normalisation des pixels entre -1 et 1.

2. Construction du modèle

- **Générateur** : Basé sur un réseau encodeur-décodeur utilisant des blocs RésNet.
- **Discriminateur** : Basé sur des réseaux convolutifs simples.

3. Entraînement du modèle

- Entraîné sur des GPU haute performance comme A100.
- Utilisé une stratégie progressive avec des pertes de cohérence cyclique et d'identité.
- Affichage des pertes avec une barre de progression `tqdm`.

4. Génération et animation

- Génération d'images stylisées.
- Interpolation entre les images originales et transformées.
- Sauvegarde des animations en GIF ou en vidéo MP4.

Résultats

Qualité des images générées

Les peintures générées imitent le style artistique de Monet tout en préservant les formes originales des photos.

Animation et visualisation

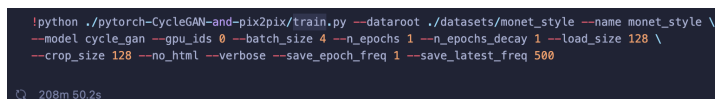
Des animations fluides montrent la transformation progressive des photos vers les peintures de Monet.

Difficultés rencontrées

Problème des ressources machines :

Au début, le projet a été réalisé sur un Mac de dernière génération. Cependant, l'absence d'un GPU performant a causé des temps d'entraînement extrêmement longs, **atteignant près de 200 heures pour une seule époque**.

Cela a rendu impossible un entraînement efficace. Voici une illustration du problème :



```
!python ./pytorch-CycleGAN-and-pix2pix/train.py --dataroot ./datasets/monet_style --name monet_style \
--model cycle_gan --gpu_ids 0 --batch_size 4 --n_epochs 1 --n_epochs_decay 1 --load_size 128 \
--crop_size 128 --no_html --verbose --save_epoch_freq 1 --save_latest_freq 500
Q 208m 50.2s
```

Illustration :

Entraînement lent sur un Mac.

Pour surmonter cette limitation, j'ai opté pour un abonnement Google Colab Pro avec un accès à des GPU (comme l'A100), ce qui a considérablement accéléré les temps d'entraînement à quelques minutes par époque.

Conclusion

Ce projet montre le potentiel des modèles de transformation de style basés sur CycleGAN. Bien qu'il soit possible d'améliorer la qualité des résultats en augmentant les époques ou en ajustant les hyperparamètres, les résultats initiaux sont très prometteurs.

Code source

Le code complet est accessible sur [CycleGAN GitHub](#).